ĐẠI HỌC QUỐC GIA THÀNH PHỐ HỒ CHÍ MINH
TRƯỜNG ĐẠI HỌC BÁCH KHOA
KHOA KHOA HỌC VÀ KỸ THUẬT MÁY TÍNH

HỌC SÂU VÀ ỨNG DỤNG TRONG THỊ GIÁC MÁY TÍNH

PHÂN LOẠI ẢNH VỚI CÁC MÔ HÌNH HỌC SÂU CƠ BẢN

Giảng viên hướng dẫn: TS. Lê Thành Sách

Sinh viên thực hiện:

2470077	Trần Phan Quốc Đạt
2470076	Nguyễn Quốc Bảo
2570307	Lê Minh Quân
2470753	Phạm Duy Tùng

TP.HCM, 2026

MỤC LỤC

I. Xây dựng các mô hình phân loại 4

1. Mô hình Softmax regression 4

2. Mô hình MLP 4

3. Mô hình CNN based 5

4. Mô hình ViT đơn giản 6

II. Huấn luyện, đánh giá và so sánh 6

1. Mô tả tập dữ liệu 6

2. Thực hiện huấn luyện và so sánh trên các mô hình cơ bản 7

III. Tự hiện thực Transformer Encoder và ViT 9

1. . Kiến trúc mô hình 9

2. Tự thực hiện khối Transformer Encoder 10

3. Huấn luyện, đánh giá và so sánh 12

IV. Các kiến trúc kết hợp và cách embed ảnh khác nhau 15

1. Tập dữ liệu 15

2. Data Augmentation 15

3. Các kiến trúc được triển khai 16

4. Kết quả thực nghiệm 19

5. Phân tích và thảo luận 23

V. Mô hình phân loại dựa trên LSTM/GRU 25

1. Mô hình chung: LSTMClassifier 25

2. Xử lý dữ liệu (Data Processing & DataLoader) 26

3. Quá trình huấn luyện 27

4. Kết quả và Đánh giá 27

5. Nhận xét và Kết luận 29

TÀI LIỆU THAM KHẢO 30

DANH MỤC HÌNH ẢNH

Hình 1. Sơ đồ phân loại dùng Softmax Regression 4
Hình 2. Sơ đồ phân loại dùng MLP 4
Hình 3. Sơ đồ phân loại dùng CNN 5
Hình 4. Sơ đồ phân loại dùng ViT 6
Hình 5. Phân phối dữ liệu trên tập CIFAR – 10 7
Hình 6. Các kết quả huấn luyện mô hình phân loại với tập dữ liệu CIFAR-10 8
Hình 7. Tổng quan mô hình ViT.....10
Hình 8. Khối Multi - head Self Attention 11
Hình 9. Khối Feed-Forward.....11
Hình 10. Hai quy ước Norm-Last và Norm-First trong Transformer Encoder 12
Hình 11. Các kết quả so sánh giữa hai mô hình ViT Built – in và Vit Rewritten 14
Hình 12. Ví dụ augmentation trên cùng một ảnh đầu vào trong CIFAR-10. 15
Hình 13. Minh hoạ hai cách patch embedding: bên trái là patch không overlap (stride = 4), bên phải là patch overlap (stride = 2). 17
Hình 14. So sánh số token và số lượng tham số trainable trước khi huấn luyện..18
Hình 15. So sánh tổng quát giữa độ chính xác test, số tham số trainable và thời gian huấn luyện trung bình mỗi epoch. 21
Hình 16. Trade-off giữa độ chính xác test và số lượng tham số.....21
Hình 17. So sánh đường validation accuracy giữa bốn mô hình. 22
Hình 18.So sánh đường validation loss giữa bốn mô hình. 23
Hình 19. Confusion matrix của mô hình tốt nhất và yếu nhất.....24
Hình 20. Kiến trúc tổng quan LSTM 25

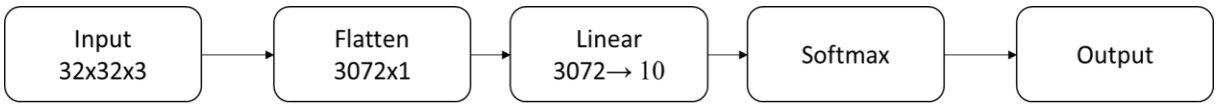
DANH MỤC BẢNG BIỂU

Bảng 1. Thông số huấn luyện mô hình ViT 13
Bảng 2. Kết quả huấn luyện Rewritten ViT và Built-in ViT 13
Bảng 3. So sánh giữa 4 mô hình embeding ảnh và kiến trúc kết hợp 20
Bảng 4. Thông số huấn luyện mô hình LTSM 26
Bảng 5. Bảng các siêu thông số của quá trình huấn luyện 27

I. Xây dựng các mô hình phân loại

Với mục tiêu nắm vững nền tảng và đạt kết quả tối ưu trong bài tập này, nhóm sẽ trình bày chi tiết cấu trúc các mô hình phân loại mà nhóm đã thiết kế, bao gồm: Softmax Regression, MLP, Simple CNN và Vision Transformer (ViT). Việc phân tích kỹ thuật các mô hình từ đơn giản đến phức tạp này sẽ tạo cơ sở vững chắc cho phần thực nghiệm, so sánh hiệu năng trên tập dữ liệu CIFAR-10 ở mục tiếp theo.

1. Mô hình Softmax regression



Hình 1. Sơ đồ phân loại dùng Softmax Regression

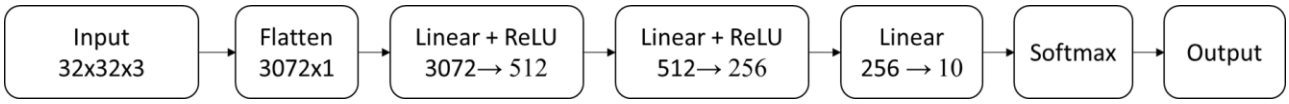
Sơ đồ trên mô tả kiến trúc của mô hình **Softmax Regression** được xây dựng bằng PyTorch, dùng để phân loại ảnh trên tập dữ liệu CIFAR-10 với 10 lớp đối tượng khác nhau. Mô hình hoạt động theo luồng xử lý gồm ba bước chính:

Bước 1: Lớp Flatten có nhiệm vụ trải phẳng tensor 3 chiều thành một vector 1 chiều kích thước (B, 3072), trong đó B là kích thước batch. Đây là bước chuẩn bị dữ liệu cần thiết trước khi đưa vào lớp tuyến tính.

Bước 2: Lớp tuyến tính thực hiện phép biến đổi $y = xW^T + b$, trong đó ma trận trọng số W có kích thước (10×3072) và vector bias b có kích thước (10,). Lớp này ánh xạ vector đặc trưng 3.072 chiều xuống còn 10 giá trị tương ứng với 10 lớp phân loại.

Bước 3: Softmax Regression để chuyển vector số học sang một phân phối xác suất. Output là vector logits kích thước (B, 10), đại diện cho xác suất từng lớp. Trong quá trình huấn luyện, hàm Softmax thường được tích hợp ngầm vào hàm mất mát CrossEntropyLoss thay vì gọi tường minh trong forward().

2. Mô hình MLP



Hình 2. Sơ đồ phân loại dùng MLP

Bước 1: Lớp Flaten trải phẳng tensor 3 chiều thành vector 1 chiều kích thước (B, 3072), trong đó 3072 = 32×32×3. Đây là bước chuẩn bị cần thiết trước khi đưa dữ liệu

vào các lớp tuyến tính.

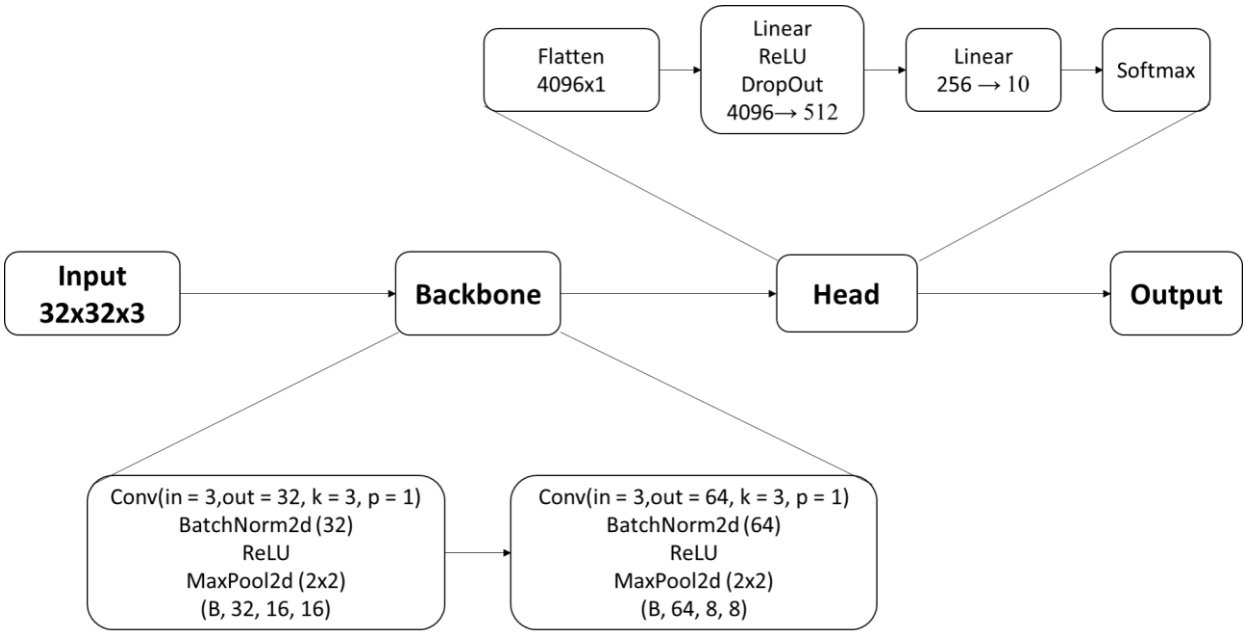
Bước 2: Linear + ReLU (3072 → 512) thực hiện phép biến đổi y = xW^T + b, với W có kích thước (512×3072). Sau đó, hàm kích hoạt ReLU được áp dụng để đưa phi tuyến vào mô hình, giúp học đặc trưng mức thấp.

Bước 3: Linear + ReLU (512 → 256) tiếp tục biến đổi đặc trưng qua lớp tuyến tính (kích thước 256×512), kết hợp ReLU để học các đặc trưng trừu tượng hơn và giảm chiều dữ liệu.

Bước 4: Linear (256 → 10) g ánh xạ vector đặc trưng 256 chiều thành 10 giá trị (logits), tương ứng với 10 lớp phân loại.

Bước 5: Hàm Softmax chuyển vector logits thành phân phối xác suất (B, 10), trong đó tổng các xác suất bằng 1.

3. Mô hình CNN based



Hình 3. Sơ đồ phân loại dùng CNN

Bước 1: Ảnh đi qua lớp tích chập với 32 bộ lọc (kernel 3×3, padding = 1), sau đó được chuẩn hóa bằng BatchNorm, kích hoạt ReLU và giảm kích thước bằng MaxPooling 2×2, thu được tensor (B, 32, 16, 16).

Bước 3: Tiếp tục qua lớp tích chập với 64 bộ lọc, kết hợp BatchNorm và ReLU để học đặc trưng sâu hơn, rồi MaxPooling 2×2 giảm kích thước còn (B, 64, 8, 8).

Bước 4: Tensor đặc trưng được trải phẳng thành vector 1 chiều kích thước (B,

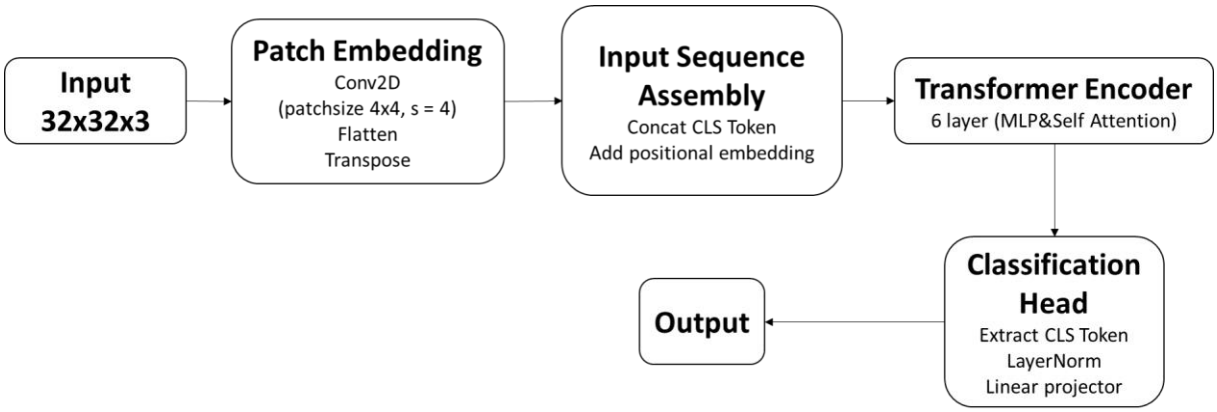
4096) để đưa vào các lớp fully connected.

Bước 5: Lớp tuyến tính giúp học đặc trưng tổng hợp từ kích thước 4096 về 512, ReLU tăng tính phi tuyến, và Dropout giúp giảm overfitting trong quá trình huấn luyện.

Bước 6: Vector đặc trưng được ánh xạ thành 10 giá trị ứng với 10 lớp phân loại.

Bước 7: Các logits được chuyển thành xác suất, giúp mô hình so sánh và đưa ra dự đoán.

4. Mô hình ViT đơn giản



Hình 4. Sơ đồ phân loại dùng ViT

Bước 1: Ảnh được chia thành các patch nhỏ (ví dụ 4×4), sau đó mỗi patch được chuyển thành vector đặc trưng thông qua Conv2d/Linear Projection, tạo thành chuỗi (B, N, D).

Bước 2: Thêm một CLS Token vào đầu chuỗi để đại diện toàn ảnh, đồng thời cộng Positional Embedding để giữ thông tin vị trí các patch.

Bước 3: Chuỗi vector đi qua 6 lớp Transformer, gồm Self-Attention và MLP, giúp học quan hệ toàn cục giữa các patch.

Bước 4: Lấy vector CLS Token, chuẩn hóa bằng LayerNorm và đưa qua Linear để phân loại.

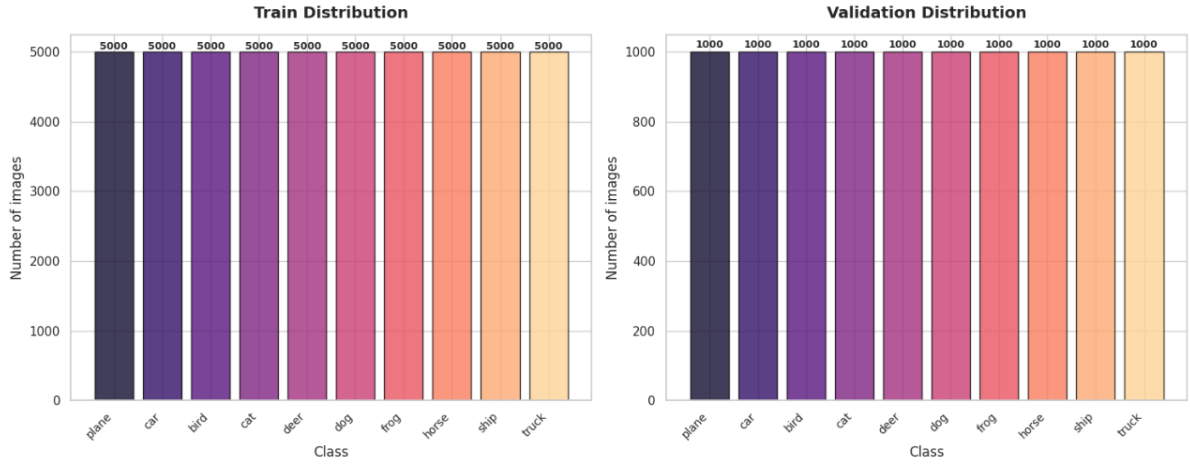
II. Huấn luyện, đánh giá và so sánh

1. Mô tả tập dữ liệu

Tập dữ liệu được sử dụng là CIFAR-10 gồm ảnh màu kích thước 32×32. Dữ liệu được tách theo kiểu stratified split: 50.000 ảnh cho train, 10.000 ảnh cho validation.

Số lớp: 10 lớp (máy bay, ô tô, chim, mèo, chó,…).

Link dataset: https://www.cs.toronto.edu/~kriz/cifar.html



Hình 5. Phân phối dữ liệu trên tập CIFAR – 10

2. Thực hiện huấn luyện và so sánh trên các mô hình cơ bản

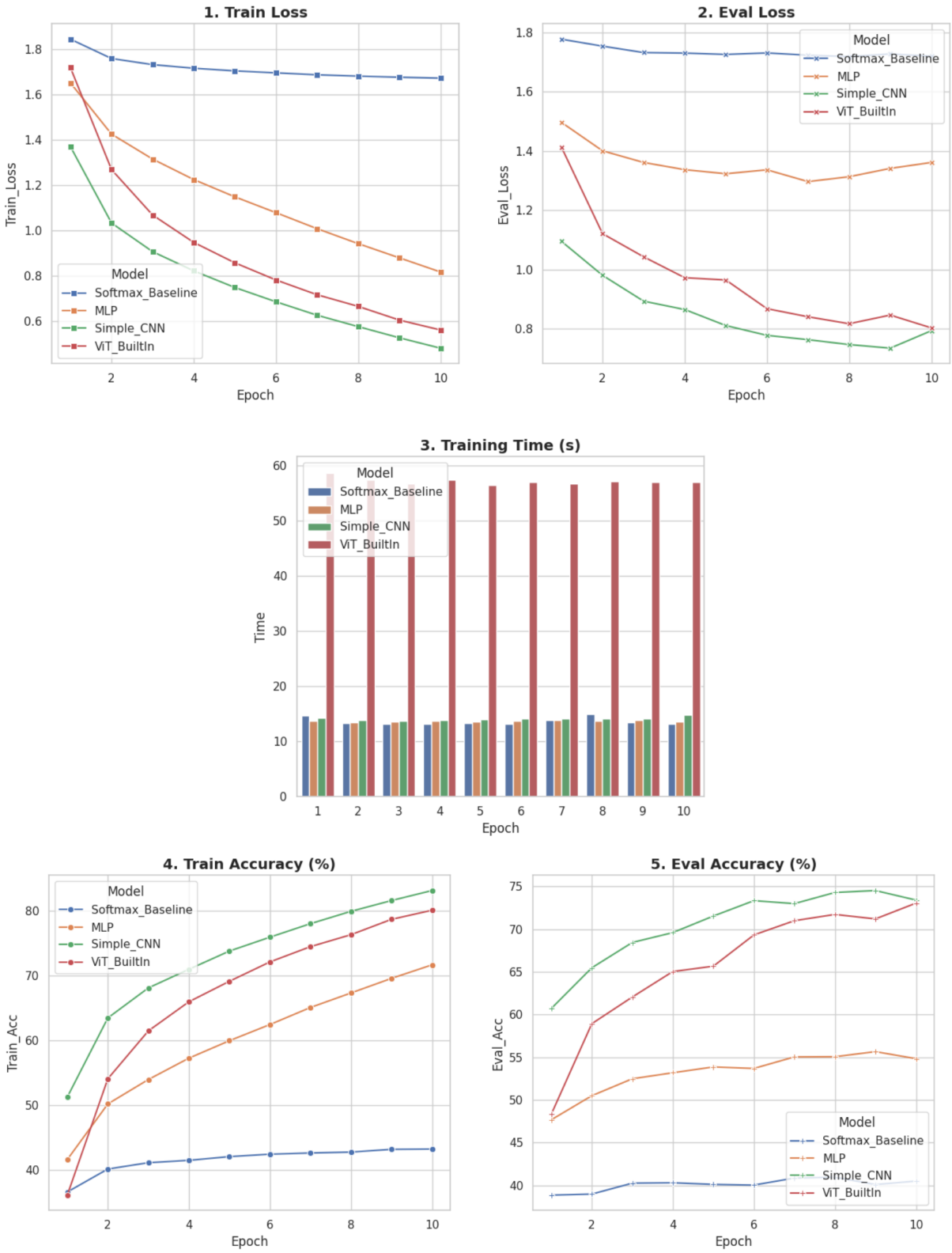
Sau khi nhóm thực hiện training, huấn luyện và đánh giá trên tập CIFAR - 10 thu được kết quả như các hình bên dưới.

Nhìn chung, có sự phân hóa rõ rệt về hiệu năng giữa các kiến trúc. Simple_CNN và ViT_BuiltIn vượt trội hoàn toàn so với hai mô hình cơ bản là Softmax và MLP. Điều này chứng minh rằng đối với dữ liệu hình ảnh, các kiến trúc có khả năng trích xuất đặc trưng không gian (CNN) hoặc học quan hệ toàn cục (Transformer) hiệu quả hơn nhiều so với các mạng kết nối đầy đủ (Fully Connected) đơn thuần.

a) Về kết quả Accuracy & Loss:

- Simple_CNN: Đây là mô hình hoạt động hiệu quả nhất trong bài toán này. Nó đạt độ chính xác cao nhất trên tập Eval (~74%) và có đường Loss giảm ổn định nhất. Việc CNN chiếm ưu thế là dễ hiểu vì CIFAR-10 gồm các ảnh nhỏ, nơi đặc trưng cục bộ đóng vai trò then chốt.
- ViT_BuiltIn: Đứng vị trí thứ hai. Dù ở những Epoch đầu tiên khởi đầu khá chậm (thấp hơn cả MLP về Accuracy ở Epoch 1), nhưng sau đó ViT học rất nhanh và tiệm cận sát nút với CNN. Điều này cho thấy Transformer cần một thời gian "khởi động" để học cách chú ý vào các vùng ảnh.
- MLP: Đạt mức trung bình (~55% Eval Accuracy). Dù tốt hơn Baseline nhưng MLP bộc lộ giới hạn rõ rệt khi đường Eval Loss có dấu hiệu đi ngang hoặc tăng nhẹ ở các Epoch cuối, cho thấy dấu hiệu của Overfitting.

- Softmax_Baseline: Hiệu năng thấp nhất (~40% Accuracy). Mô hình này quá đơn giản, không đủ khả năng học các đặc trưng phức tạp của ảnh màu, dẫn đến tình trạng Underfitting.



Hình 6. Các kết quả huấn luyện mô hình phân loại với tập dữ liệu CIFAR-10

b) Thời gian huấn luyện

- ViT_BuiltIn: Thời gian huấn luyện mỗi Epoch lên tới gần 60 giây, cao gấp khoảng 4-5 lần so với các mô hình còn lại. Điều này là do cơ chế Self-Attention tính toán ma trận tương quan giữa tất cả các patch ảnh, đòi hỏi chi phí tài nguyên rất lớn.
- Simple_CNN, MLP, Softmax: Cả ba đều có thời gian huấn luyện tương đồng và rất nhanh (khoảng 13-15 giây mỗi Epoch). Trong đó, CNN cho hiệu năng tốt nhất trên mỗi đơn vị thời gian.

III. Tự hiện thực Transformer Encoder và ViT

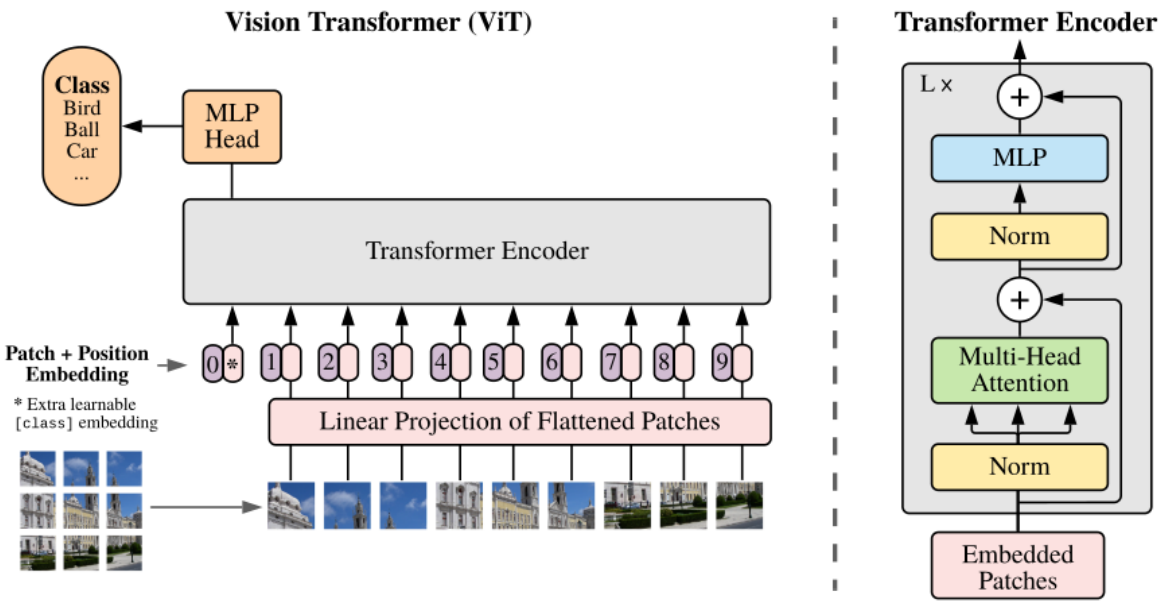
Để bắt đầu với quá trình tự hiện thực mô hình transformer chúng ta cần nhìn lại kiến trúc tổng quát của mô hình, cách xây dựng và các thành phần cơ bản của mô hình. Việc hiểu biết rõ sẽ giúp thiết kế mô hình được chuẩn và chặt chẽ hơn, từ đó tối ưu hóa được hiệu suất tính toán.

1. . Kiến trúc mô hình

Các thành phần chính của mô hình ViT gồm có:

- Patch Embedding: chia ảnh đầu vào thành các patch với kích thước cố định, làm phẳng thành một chuỗi rồi nhúng tuyến tính các patch vào không gian đặc trưng.
- Positional Embedding: cộng các embedding vị trí vào để thể hiện sự quan trọng về vị trí. Ngoài ra, class token được thêm vào ở embedding đầu tiên để phân loại.
- Transformer Encoder: học các mối quan hệ giữa các patch thông qua cơ chế self-attention. Ngoài ra, nó cũng học thông tin toàn bộ ảnh thông qua class token.
- MLP Head: đầu phân loại gồm một lớp tuyến tính để giảm số chiều của embedding bằng với số lớp rồi qua hàm SoftMax để phân loại.

Tóm lại, việc hiện thực mô hình Vision Transformer (ViT) là quá trình chuyển đổi tư duy xử lý hình ảnh từ dạng lưới pixel cục bộ sang dạng chuỗi dữ liệu có tính quan hệ toàn cầu. Đặc điểm này giúp ViT khắc phục được nhược điểm của CNN khi ở những lớp đầu tiên mô hình ViT có thể thấy được đặc trưng toàn cục của toàn bộ ảnh.



Hình 7. Tổng quan mô hình ViT

2. Tự thực hiện khối Transformer Encoder

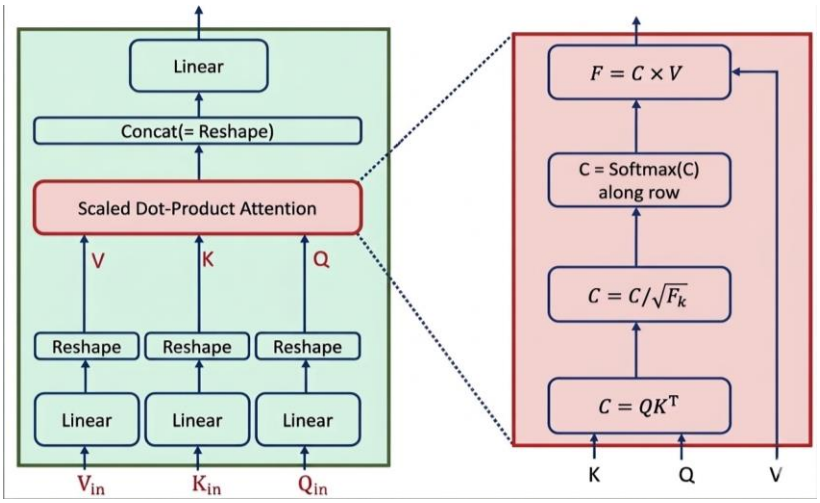
Transformer Encoder: bao gồm nhiều lớp con, trong mỗi lớp con chứa hai khối chính Multi-head Self Attention và Feed-Forward Network. Ngoài ra, nó còn kết hợp thêm các khối Residual Connection và Layer Normalization, mặc dù về mặt khái niệm là không cần thiết, nhưng lại cần thiết cho sự ổn định và hội tụ về mặt số học. Từ đó sinh ra hai quy ước phổ biến đang được sử dụng: Norm-Last và Norm-First được mô tả như các hình bên dưới.

a) Multi-head Self Attention

Sơ đồ của khối Multi Head Attention bắt đầu với ba đầu vào Vin, Kin, và Qin. Mỗi đầu vào này được đưa qua một lớp Linear riêng biệt để biến đổi sang không gian đặc trưng mới. Sau đó, các kết quả được reshape (thay đổi kích thước tensor) để phù hợp với việc tính toán attention. Kết quả của bước này là ba ma trận Q (Query), K (Key) và V (Value).

Tiếp theo, trong khối attention, ma trận Q được nhân với K chuyển vị để đo mức độ liên quan giữa các phần tử. Kết quả này tạo ra một ma trận thể hiện "độ giống nhau" giữa từng Query và Key. Những giá trị lớn biểu thị mức độ liên quan cao hơn. Ma trận độ tương đồng sau đó được chia cho căn bậc hai của kích thước vector (scale) để tránh giá trị quá lớn gây mất ổn định khi huấn luyện. Sau đó, hàm Softmax được áp dụng theo từng hàng để chuyển các giá trị này thành xác suất, thể hiện mức độ quan

trọng của từng phần tử. Các trọng số thu được từ Softmax sẽ được nhân với ma trận V. Bước này giúp tổng hợp thông tin: những phần tử quan trọng hơn sẽ đóng góp nhiều hơn vào kết quả cuối cùng. Kết quả này chính là đầu ra của khối attention.

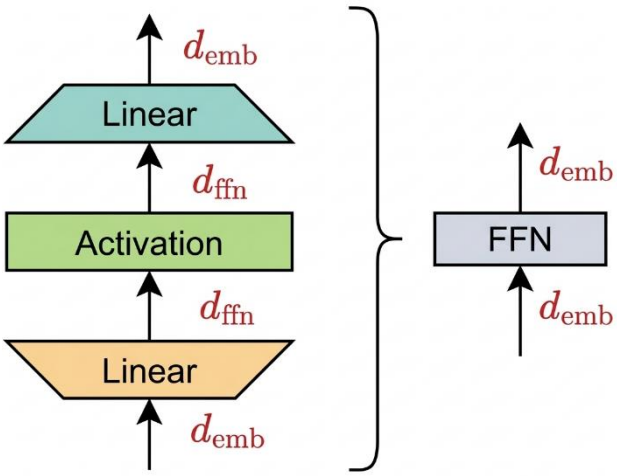


Hình 8. Khối Multi - head Self Attention

Sau khi tính attention, các đầu ra (có thể từ nhiều head) sẽ được **concat (ghép lại)** và **reshape** về kích thước ban đầu. Cuối cùng, một lớp **Linear** nữa được áp dụng để tạo ra đầu ra hoàn chỉnh của khối này.

b) Feed-Forward Network

Khối Feed Forward Network (FFN) nhận vector đầu vào kích thước d_{emb}, sau đó đưa qua hai lớp Linear: lớp đầu tiên tăng chiều lên d_{ffn}, tiếp theo là hàm Activation để học quan hệ phi tuyến, rồi lớp Linear thứ hai giảm chiều về lại d_{emb}. Mục đích của khối này là tăng khả năng biểu diễn đặc trưng cho từng phần tử dữ liệu sau khi đã qua attention.

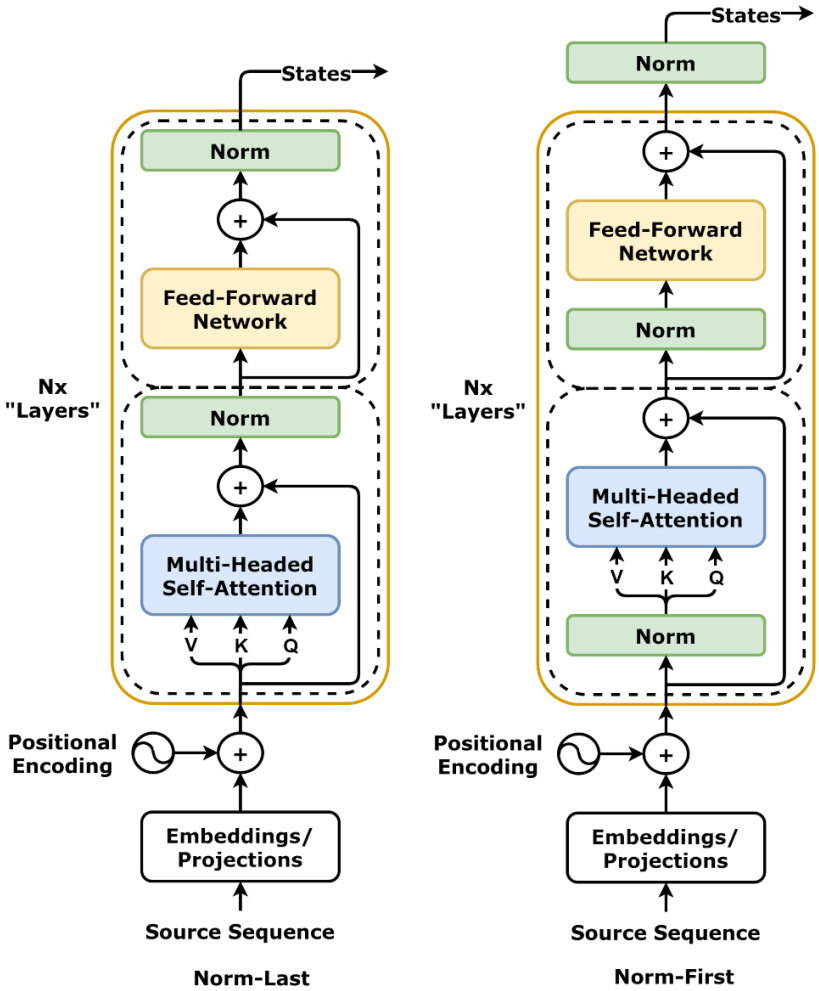


Hình 9. Khối Feed-Forward

c) Norm – First và Norm – Last

Như đã trình bày ở trên, việc tổ chức này giúp mô hình trong quá trình học được

ổn định và hội tụ. Dữ liệu đầu vào được embedding và cộng với positional encoding, sau đó đi qua nhiều lớp lặp lại gồm Multi-Head Self-Attention và Feed Forward Network (FFN). Mỗi khối đều có residual connection (cộng skip) và LayerNorm, nhưng khác nhau ở thứ tự: Norm-Last thì chuẩn hóa sau mỗi khối, còn Norm-First thì chuẩn hóa trước khi đưa vào attention và FFN.



Hình 10. Hai quy ước Norm-Last và Norm-First trong Transformer Encoder

3. Huấn luyện, đánh giá và so sánh

Dưới đây là bảng các tham số huấn luyện của mô hình ViT. Những giá trị này được lựa chọn dựa trên mục tiêu đạt được sự cân bằng hợp lý giữa hiệu năng mô hình, tốc độ huấn luyện và chi phí tài nguyên tính toán. Cụ thể, với bài toán phân loại ảnh kích thước nhỏ như CIFAR-10, việc sử dụng cấu hình vừa phải giúp mô hình vẫn học được các đặc trưng quan trọng mà không trở nên quá phức tạp hay dễ bị overfitting.

Bảng 1. *Thông số huấn luyện mô hình ViT*

Tham số	Giá trị	Chú thích
EMBED_DIM	256	Kích thước của embedding
N_HEADS	8	Số lượng đầu của Attention
DEPTH	6	Số lượng các lớp con encoder
MLP_RATIO	4.0	Tỉ lệ độ rộng lớp ẩn trong khối FFN
ACTIVATION	ReLU	Hàm activate trong khối FFN
NORM_FIRST	False	Dùng Norm-Last
DROPOUT	0.1	Xác suất dropout

Kết quả mô hình sau khi training giữa mô hình ViT có sẵn và ViT tự viết như sau:

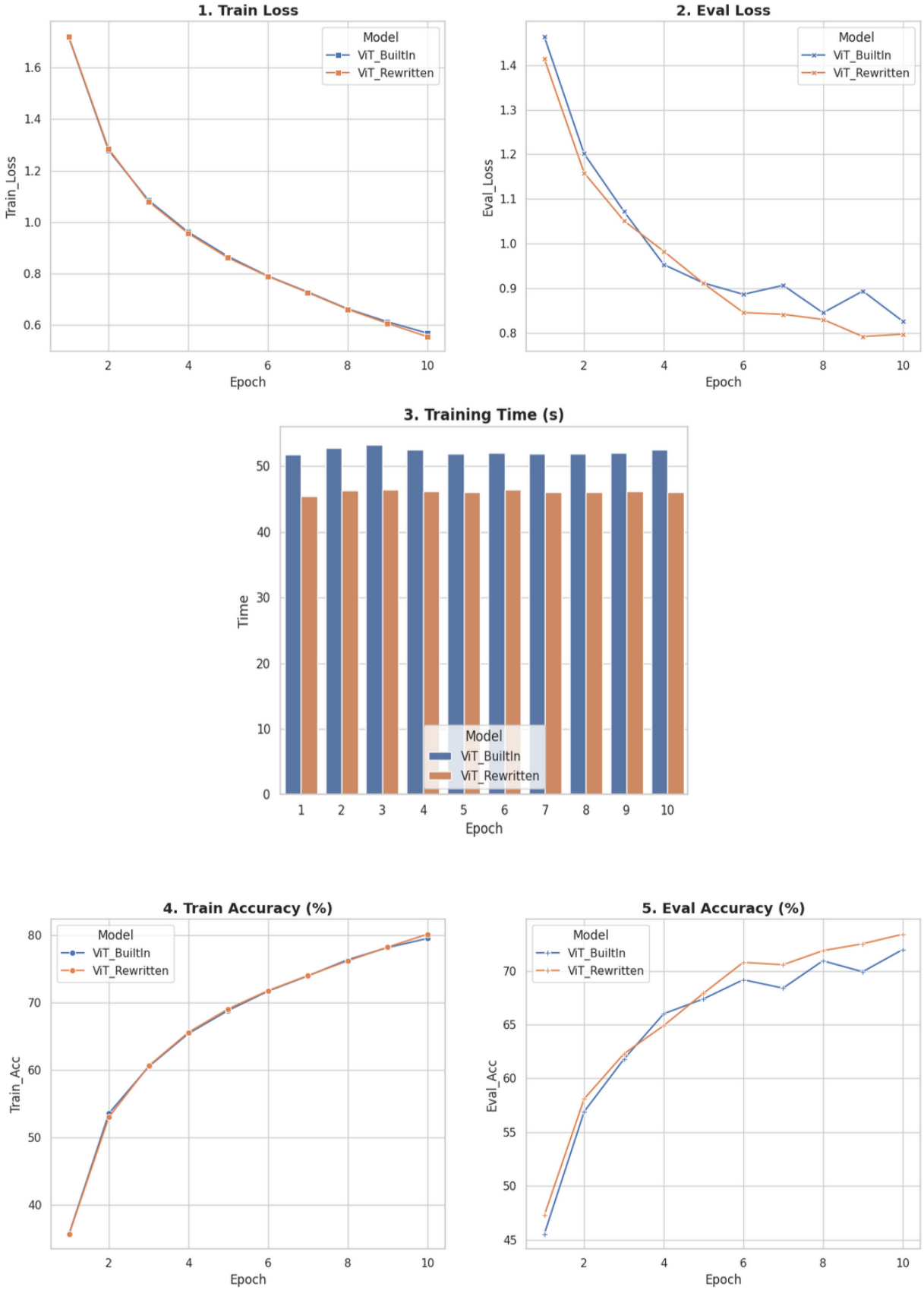
Bảng 2. *Kết quả huấn luyện Rewritten ViT và Built-in ViT*

Tiêu chí	**Rewritten ViT**	**Built-in ViT**
Accuracy	73.4%	72.0%
Hàm Loss	0.797	0.826
Thời gian/Epoch	~46s	~52s

Hai mô hình đạt hiệu năng khá tương đồng, với độ chính xác lần lượt là 73.4% và 72.0%, cùng giá trị loss tương ứng là 0.797 và 0.826 đối với Rewritten ViT và Built-in ViT. Mặc dù chênh lệch về mặt hiệu năng không đáng kể, mô hình Rewritten ViT lại cho thấy ưu thế về thời gian huấn luyện khi chỉ mất khoảng 46 giây, nhanh hơn so với khoảng 52 giây của mô hình Built-in. Sự khác biệt này có thể xuất phát từ việc phiên bản Rewritten đã được tinh giản, lược bỏ một số bước kiểm tra và cơ chế tổng quát hóa được tích hợp sẵn trong PyTorch.

Bên cạnh đó, xét về quá trình huấn luyện, Rewritten ViT có xu hướng dao động loss và accuracy hơn giữa các epoch gần cuối, trong khi mô hình Built-in ViT thể hiện sự ổn định và mượt mà hơn trong quá trình hội tụ. Nhìn chung, cả hai mô hình đều đạt hiệu quả tốt, nhưng Built-in ViT tỏ ra phù hợp hơn trong các kịch bản cần tối ưu thời

gian và tài nguyên.



Hình 11. Các kết quả so sánh giữa hai mô hình ViT Built – in và Vit Rewritten

IV. Các kiến trúc kết hợp và cách embed ảnh khác nhau

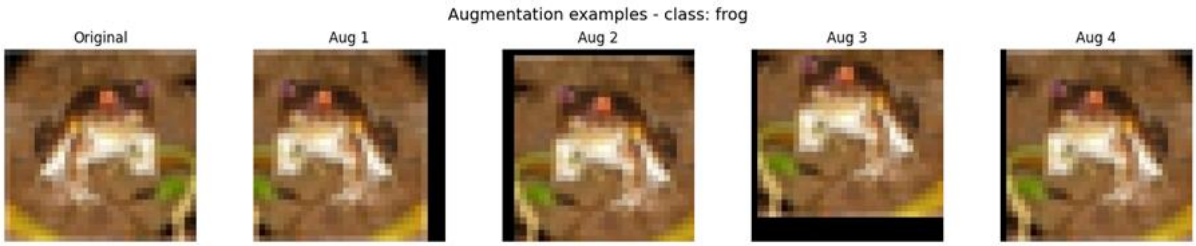
Mục tiêu của phần này là khảo sát ảnh hưởng của cách biểu diễn ảnh thành token và cách kết hợp CNN với Transformer đối với bài toán phân loại ảnh trên CIFAR-10. Thay vì chỉ thử một kiến trúc, nhóm sẽ triển khai nhiều hướng khác nhau để trả lời ba câu hỏi chính: (i) Dùng backbone CNN rồi đưa token vào Transformer có hiệu quả hơn ViT thuần hay không? (ii) Patch overlap có tốt hơn patch không overlap hay không? và (iii) Cách coi chiều không gian H×W là token có hợp lý hơn cách coi mỗi kênh đặc trưng C là một token hay không?

1. Tập dữ liệu

Tập dữ liệu được sử dụng là CIFAR-10, gồm 10 lớp ảnh màu kích thước 32×32. Notebook thực hiện tách dữ liệu theo kiểu stratified split: 45.000 ảnh cho train, 5.000 ảnh cho validation và 10.000 ảnh cho test.

2. Data Augmentation

Tiền xử lý và augmentation áp dụng như sau: train dùng RandomHorizontalFlip, RandomCrop(32, padding=4), ToTensor và Normalize; validation/test dùng ToTensor và Normalize với mean/std chuẩn của CIFAR-10. Tất cả mô hình đều được huấn luyện trong 12 epoch với batch size 128, AdamW (learning rate 1e-3, weight decay 1e-4), scheduler CosineAnnealingLR, và cùng một training loop tự viết bằng PyTorch.



Hình 12. Ví dụ augmentation trên cùng một ảnh đầu vào trong CIFAR-10.

Ngoài việc dữ liệu train đã cân bằng, nhóm vẫn áp dụng data augmentation trong giai đoạn huấn luyện để tăng tính đa dạng của dữ liệu đầu vào và cải thiện khả năng khái quát hóa của mô hình. Cụ thể, trong notebook nhóm sử dụng hai phép biến đổi đơn giản gồm lật ngang ngẫu nhiên (RandomHorizontalFlip) và cắt ngẫu nhiên có đệm biên (RandomCrop(32, padding=4)). Các phép biến đổi này được áp dụng trực tiếp trong quá trình lấy batch huấn luyện, tức là ảnh gốc không bị thay đổi vĩnh viễn trong dataset, mà

mỗi epoch mô hình có thể nhìn thấy những biến thể khác nhau của cùng một ảnh. Hình 12 minh họa một số phiên bản augmentation sinh ra từ cùng một ảnh đầu vào.

Việc sử dụng augmentation ở đây không nhằm xử lý mất cân bằng lớp, mà chủ yếu nhằm giảm overfitting và giúp mô hình học được các đặc trưng bền vững hơn trước những thay đổi nhỏ về vị trí và hướng của đối tượng trong ảnh. Đây là lựa chọn phù hợp với CIFAR-10, vì dữ liệu có kích thước nhỏ và các mô hình Transformer/hybrid trong Phần 4 khá nhạy với chất lượng biểu diễn đầu vào. Ngoài ra, để đảm bảo đánh giá khách quan, nhóm không áp dụng augmentation cho validation và test set; hai tập này chỉ được chuẩn hóa bằng thống kê mean và standard deviation của CIFAR-10.

3. Các kiến trúc được triển khai

Ở phần này nhóm sẽ triển khai 4 kiến trúc khác nhau. Các mô hình dùng cùng Transformer classifier core, khác nhau chủ yếu ở cách tạo token đầu vào.

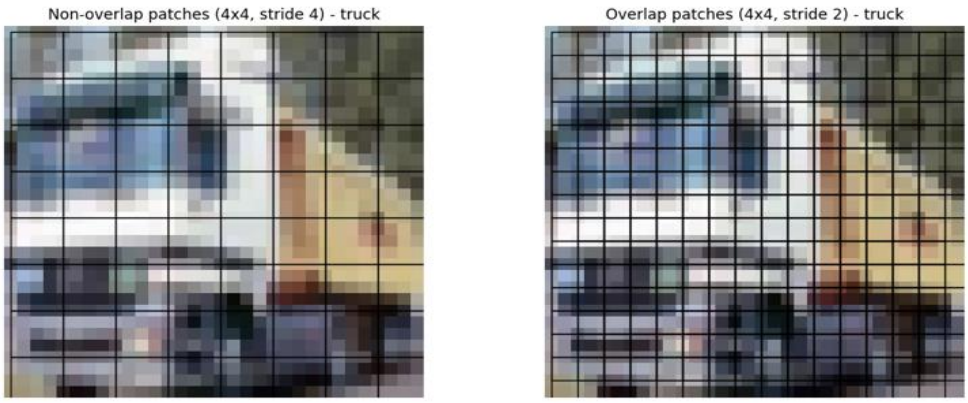
Mô hình	Ý tưởng chính	Kiểu token	Số token
HybridCNN SpatialTokens	CNN backbone tạo feature map 8×8×128; mỗi vị trí không gian là một token, sau đó chiếu tuyến tính sang embed_dim = 96 rồi đưa vào Transformer.	Spatial tokens từ feature map CNN	64
ViTNonOverlap	Ảnh được chia patch 4×4 với stride = 4, tức patch không chồng lắp.	Patch tokens không overlap	64
ViTOverlap	Ảnh được chia patch 4×4 với stride = 2, tức patch có chồng lắp nên giữ lại nhiều thông tin biên hơn.	Patch tokens có overlap	225
HybridCNN ChannelTokens	CNN backbone tạo feature map 8×8×128; mỗi kênh đặc trưng được xem là một token, vector 8×8 được flatten rồi chiếu sang embed_dim = 96.	Channel tokens từ feature map CNN	128

Bốn kiến trúc trên được thiết kế sao cho phần Transformer classifier core được giữ gần như thống nhất, còn khác biệt chính nằm ở cách tạo token đầu vào. Cách thiết

kế này giúp quá trình so sánh trở nên công bằng hơn, vì khi hiệu năng thay đổi, nguyên nhân chủ yếu có thể quy về chiến lược biểu diễn ảnh thay vì do thay đổi quá nhiều thành phần trong backbone hoặc head phân loại. Nói cách khác, đây là một thiết lập thực nghiệm phù hợp với mục tiêu của Phần IV là khảo sát ảnh hưởng của kiến trúc kết hợp và cách embed ảnh khác nhau.

Hai mô hình ViTNonOverlap và ViTOverlap đại diện cho hướng tiếp cận patch embedding trực tiếp từ ảnh gốc. Trong đó, ViTNonOverlap chia ảnh thành các patch rời nhau kích thước 4×4 với stride = 4, nên mỗi patch đóng vai trò như một token độc lập. Cách làm này đơn giản, chi phí tính toán thấp hơn và gần với Vision Transformer cơ bản. Ngược lại, ViTOverlap sử dụng patch 4×4 nhưng với stride = 2, tạo ra các patch có chồng lắp. Nhờ vậy, thông tin biên giữa các vùng ảnh được giữ lại nhiều hơn, song đổi lại số token tăng mạnh, làm chi phí attention và thời gian huấn luyện lớn hơn.

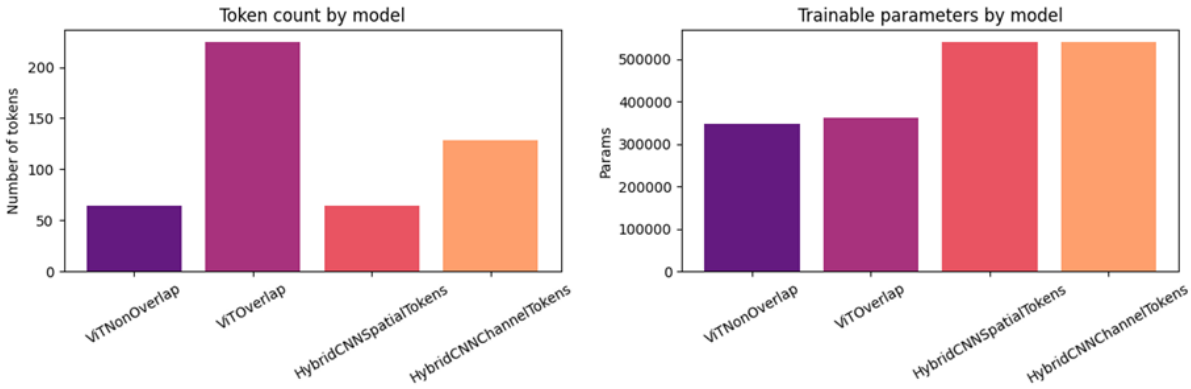
Hai mô hình HybridCNNSpatialTokens và HybridCNNChannelTokens đại diện cho hướng kết hợp CNN với Transformer. Ở đây, CNN backbone trước tiên học đặc trưng cục bộ từ ảnh, sau đó feature map được chuyển đổi thành chuỗi token theo hai cách khác nhau. Với HybridCNNSpatialTokens, mỗi vị trí không gian trên feature map 8×8 được xem là một token, vì vậy mô hình học quan hệ giữa các vùng không gian đã được CNN mã hóa trước. Trong khi đó, HybridCNNChannelTokens xem mỗi kênh đặc trưng là một token; mỗi token khi đó mang ý nghĩa là một kiểu phản hồi đặc trưng khác nhau của CNN trên toàn bộ không gian 8×8. Đây là một biến thể thú vị vì nó thay đổi hoàn toàn "góc nhìn" của Transformer: thay vì mô hình hóa quan hệ giữa các vị trí ảnh, nó mô hình hóa quan hệ giữa các kênh đặc trưng.



Hình 13. Minh hoạ hai cách patch embedding: bên trái là patch không overlap (stride = 4), bên phải là patch overlap (stride = 2).

Từ góc độ thiết kế, bốn mô hình trải đều trên ba hướng mà đề bài gợi ý: CNN + Transformer, patch/tokenizer khác nhau, và thay đổi định nghĩa token theo chiều không gian hoặc chiều kênh. Do đó, tập mô hình này không chỉ đáp ứng đúng yêu cầu của đề mà còn tạo ra cơ sở hợp lý để phân tích sâu hơn về sự đánh đổi giữa độ chi tiết biểu diễn, số lượng token, độ phức tạp mô hình và hiệu năng phân loại.

Hình 13 minh họa trực quan sự khác biệt giữa hai cách patch embedding là không overlap và có overlap. Ở phía trái, với stride = 4, các patch được lấy rời nhau nên một điểm ảnh chỉ thuộc về đúng một patch. Điều này giúp số token nhỏ hơn và việc tính toán attention nhẹ hơn. Tuy nhiên, nhược điểm là thông tin nằm ở ranh giới giữa hai patch có thể bị tách rời khá mạnh. Ở phía phải, với stride = 2, các patch chồng lắp lên nhau nên mỗi vùng ảnh có thể xuất hiện trong nhiều token khác nhau. Cách biểu diễn này giàu thông tin hơn, đặc biệt ở vùng biên và các chi tiết nhỏ, nhưng đồng thời làm số token tăng lên đáng kể. Đối với ảnh CIFAR-10 có kích thước chỉ 32×32, khác biệt giữa hai cách patching này là khá đáng chú ý. Khi patch không overlap, mô hình thu được số token tương đối gọn, phù hợp với huấn luyện nhanh và chi phí thấp. Ngược lại, overlap patch giúp biểu diễn mịn hơn, nhưng do self-attention có độ phức tạp tăng theo số token, nên mô hình thường tốn tài nguyên hơn. Vì vậy, đây là một bài toán đánh đổi điển hình giữa mức độ chi tiết của biểu diễn đầu vào và hiệu quả tính toán.



Hình 14. So sánh số token và số lượng tham số trainable trước khi huấn luyện.

Hai biểu đồ cột cho thấy sự khác biệt rõ giữa các mô hình về số lượng token và số tham số học được. Trước hết, xét về số token, ViTNonOverlap và HybridCNNSpatialTokens cùng tạo ra 64 token, nhưng bản chất hai loại token này khác nhau: mô hình ViT lấy token trực tiếp từ patch ảnh, còn mô hình hybrid lấy token từ feature map đã qua CNN backbone. ViTOverlap có số token lớn nhất, lên tới 225 token,

do sử dụng stride nhỏ hơn khi tạo patch. Trong khi đó, HybridCNNChannelTokens sinh ra 128 token, tương ứng với số kênh đặc trưng của feature map CNN. Điều này cho thấy chỉ riêng việc thay đổi định nghĩa token đã làm thay đổi đáng kể kích thước chuỗi đầu vào của Transformer.

Về số tham số, hai mô hình hybrid có số lượng tham số lớn hơn rõ rệt so với hai mô hình ViT patch-based. Nguyên nhân chính là do phần CNN backbone bổ sung thêm nhiều lớp tích chập và batch normalization trước khi dữ liệu được đưa vào Transformer. Mặt khác, dù ViTOverlap có số token lớn hơn nhiều so với ViTNonOverlap, số tham số của hai mô hình này không chênh lệch quá lớn, vì việc tăng số token chủ yếu làm tăng chi phí tính toán trong quá trình forward/backward, chứ không làm tăng mạnh số lượng trọng số cần học. Nói cách khác, số token ảnh hưởng mạnh đến chi phí tính toán, còn số tham số phản ánh độ lớn của mô hình; hai đại lượng này có liên quan nhưng không hoàn toàn đồng nhất.

Kết quả này cũng gợi ý rằng khi đánh giá mô hình, không nên chỉ nhìn vào accuracy, mà cần xem đồng thời số token, số tham số và thời gian huấn luyện. Một mô hình có thể đạt kết quả tốt hơn nhờ biểu diễn đầu vào chi tiết hơn, nhưng đổi lại phải chấp nhận chi phí tính toán lớn hơn. Đây chính là một trong những trọng tâm cần phân tích ở phần kết quả thực nghiệm của mục này.

Từ các phân tích trên có thể thấy bốn kiến trúc không chỉ khác nhau về mặt ý tưởng, mà còn khác nhau khá rõ về mức độ nén thông tin đầu vào trước khi đưa vào Transformer. Vì vậy, phần thực nghiệm tiếp theo sẽ tập trung trả lời ba câu hỏi chính: (1) cách tạo token nào cho kết quả phân loại tốt nhất trên CIFAR-10; (2) việc tăng số token có thực sự cải thiện hiệu năng hay chỉ làm tăng chi phí tính toán; và (3) mô hình hybrid CNN–Transformer có mang lại lợi thế rõ rệt so với các mô hình patch-based hay không.

4. Kết quả thực nghiệm

Bảng dưới đây trình bày kết quả sau khi huấn luyện 4 mô hình gồm HybridCNNSpatialTokens, HybridCNNChannelTokens, ViTOverlap và ViTNonOverlap. Các chỉ số được báo cáo gồm số lượng token đầu vào, số tham số trainable, best validation accuracy, test accuracy và thời gian huấn luyện trung bình trên

mỗi epoch.

Kết quả cho thấy HybridCNNSpatialTokens đạt hiệu năng cao nhất trong cả hai chỉ số đánh giá chính, với best validation accuracy = 84,94% và test accuracy = 84,85%. Đứng thứ hai là HybridCNNChannelTokens với 76,82% trên validation và 76,50% trên test. Hai mô hình ViT thuần cho kết quả thấp hơn, trong đó ViTOverlap đạt 71,52% / 71,54%, còn ViTNonOverlap là mô hình thấp nhất với 65,74% / 65,85%.

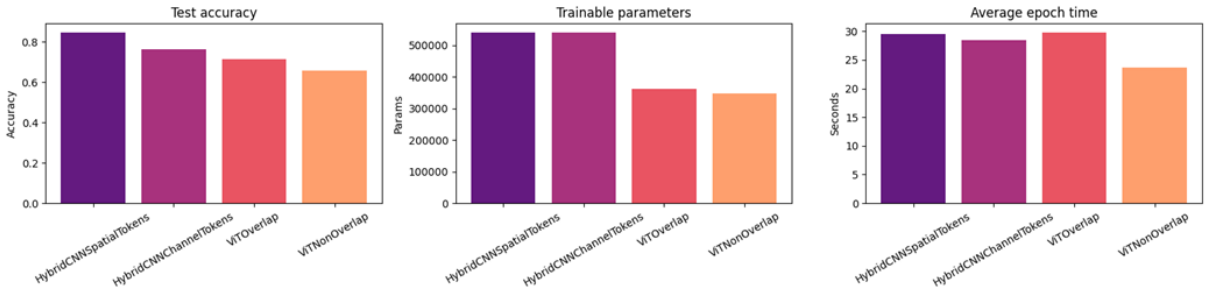
Bảng 3. *So sánh giữa 4 mô hình embeding ảnh và kiến trúc kết hợp*

Mô hình	Kiểu token	Số token	Số tham số	Validation accuracy	Test accuracy	Thời gian /epoch
HybridCNN SpatialTokens	Spatial tokens from CNN feature map	64	541.546	84.94%	84.85%	29.52 s
HybridCNN ChannelTokens	Channel tokens from CNN feature map	128	541.546	76.82%	76.50%	28.42 s
ViTOverlap	Overlap patches (4x4, stride 2)	225	363.178	71.52%	71.54%	29.83 s
ViTNon-Overlap	Non-overlap patches (4x4, stride 4)	64	347.722	65.74%	65.85%	23.65 s

Một điểm đáng chú ý là chênh lệch giữa best validation accuracy và test accuracy ở cả bốn mô hình đều rất nhỏ. Điều này cho thấy kết quả đánh giá trên tập test tương đối nhất quán với xu hướng quan sát được trên validation set, đồng thời cho thấy quá trình huấn luyện của các mô hình diễn ra ổn định, không xuất hiện sai khác lớn giữa giai đoạn chọn mô hình và giai đoạn kiểm tra cuối cùng.

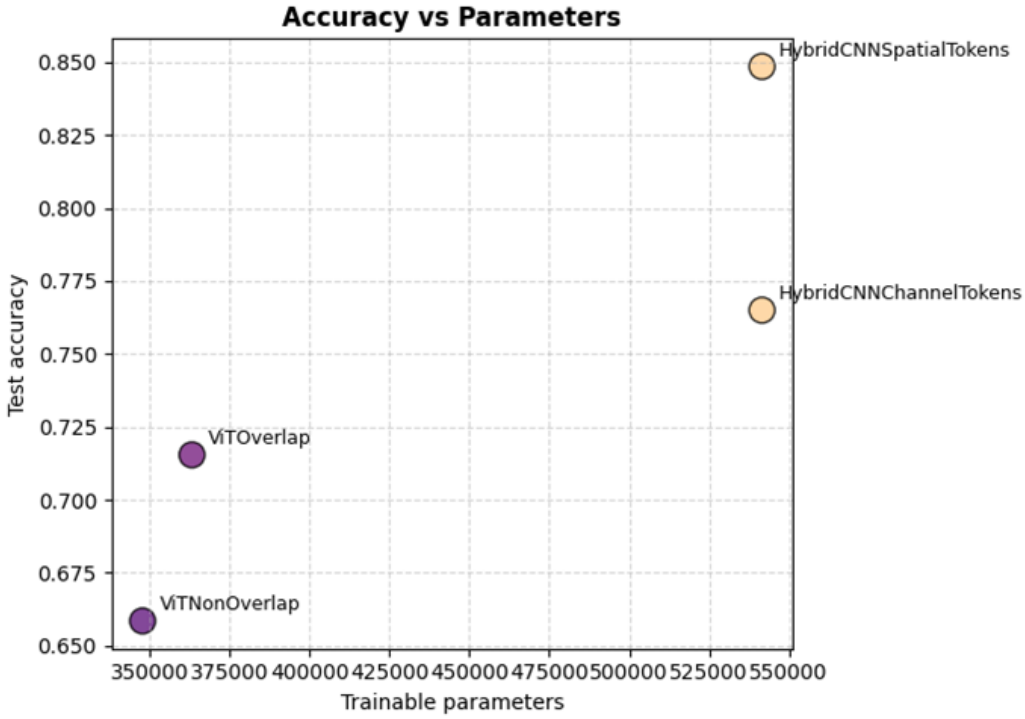
Xét về chi phí mô hình, hai kiến trúc hybrid có số tham số lớn nhất, cùng ở mức 541.546 tham số. Trong khi đó, ViTOverlap có 363.178 tham số và ViTNonOverlap có 347.722 tham số. Về thời gian huấn luyện trung bình mỗi epoch, ViTNonOverlap là mô

hình nhanh nhất với 23,65 giây/epoch, tiếp theo là HybridCNNChannelTokens với 28,42 giây/epoch, HybridCNNSpatialTokens với 29,52 giây/epoch, và ViTOverlap là mô hình chậm nhất với 29,83 giây/epoch.



Hình 15. So sánh tổng quát giữa độ chính xác test, số tham số trainable và thời gian huấn luyện trung bình mỗi epoch.

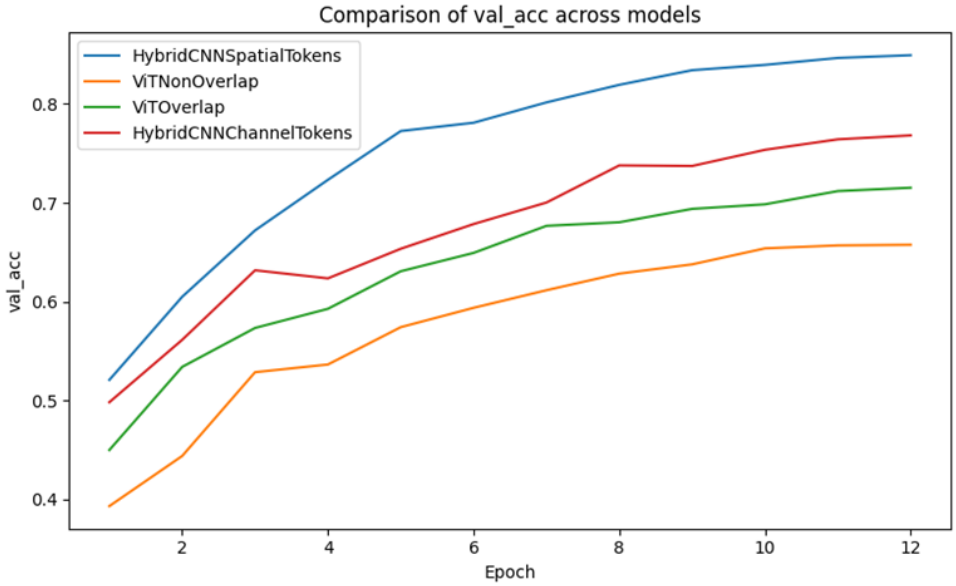
Hình 15 tổng hợp ba khía cạnh chính của bốn mô hình: độ chính xác test, số tham số và thời gian huấn luyện trung bình mỗi epoch. Quan sát từ biểu đồ cho thấy thứ hạng về test accuracy là khá rõ ràng: HybridCNNSpatialTokens cao nhất, tiếp theo là HybridCNNChannelTokens, ViTOverlap và cuối cùng là ViTNonOverlap. Đồng thời, biểu đồ tham số cho thấy hai mô hình hybrid có kích thước lớn hơn hai mô hình ViT, còn biểu đồ thời gian huấn luyện cho thấy sự khác biệt về thời gian giữa các mô hình không quá lớn, ngoại trừ ViTNonOverlap có lợi thế rõ hơn về tốc độ.



Hình 16. Trade-off giữa độ chính xác test và số lượng tham số

Hình 16 biểu diễn mối quan hệ giữa test accuracy và số tham số trainable. Các điểm dữ liệu cho thấy không có quan hệ tuyến tính đơn giản giữa hai đại lượng này. Chẳng hạn, dù HybridCNNSpatialTokens và HybridCNNChannelTokens có cùng số tham số, kết quả accuracy vẫn khác nhau đáng kể. Tương tự, ViTOverlap chỉ nhiều tham số hơn ViTNonOverlap ở mức không lớn, nhưng độ chính xác lại cao hơn rõ rệt. Biểu đồ này cho thấy việc đánh giá mô hình cần xét đồng thời cả kiến trúc và cách tạo token, thay vì chỉ dựa trên độ lớn tham số.

Diễn biến của validation accuracy theo epoch được thể hiện trong Hình 17. Tất cả các mô hình đều có xu hướng tăng dần accuracy khi số epoch tăng lên, cho thấy quá trình tối ưu diễn ra đúng hướng. HybridCNNSpatialTokens tăng nhanh ngay từ các epoch đầu và duy trì vị trí dẫn đầu trong suốt quá trình huấn luyện. HybridCNNChannelTokens cũng có đường tăng khá ổn định và luôn nằm trên hai mô hình ViT. Đối với hai mô hình patch-based, ViTOverlap duy trì kết quả tốt hơn ViTNonOverlap trong toàn bộ quá trình huấn luyện. Ở các epoch cuối, các đường cong bắt đầu chậm lại, cho thấy mô hình dần tiến tới trạng thái hội tụ.

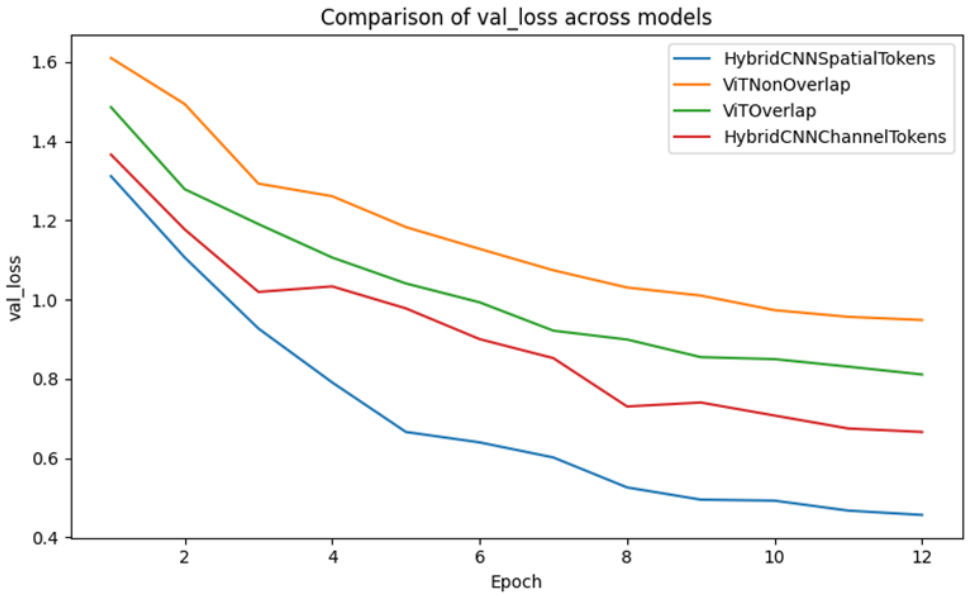


Hình 17. So sánh đường validation accuracy giữa bốn mô hình.

Song song với đó, Hình 18 cho thấy validation loss của cả bốn mô hình đều giảm đều qua các epoch. HybridCNNSpatialTokens đạt mức giảm loss nhanh nhất và duy trì giá trị thấp nhất ở cuối quá trình huấn luyện. HybridCNNChannelTokens đứng thứ hai, sau đó là ViTOverlap, còn ViTNonOverlap có loss cao nhất trong phần lớn số epoch. Xu hướng giảm loss này phù hợp với diễn biến tăng accuracy đã quan sát ở Hình 17.,

đồng thời xác nhận rằng các mô hình không chỉ tăng độ chính xác mà còn cải thiện chất lượng dự đoán một cách nhất quán theo thời gian huấn luyện.

Tổng hợp các kết quả trong mục này cho thấy bốn mô hình tạo ra sự khác biệt rõ rệt cả về hiệu năng phân loại, quy mô mô hình lẫn chi phí huấn luyện. Các kết quả định lượng và các đường cong huấn luyện là cơ sở để chuyển sang phần tiếp theo, nơi nhóm sẽ phân tích sâu hơn về nguyên nhân dẫn tới sự khác biệt giữa các kiến trúc và các cách biểu diễn token.



Hình 18. So sánh đường validation loss giữa bốn mô hình.

5. Phân tích và thảo luận

Kết quả cho thấy mô hình HybridCNNSpatialTokens đạt hiệu năng tốt nhất với best validation accuracy 84,94% và test accuracy 84,85%. Mô hình này cũng vượt rõ rệt so với ba biến thể còn lại. So với ViTNonOverlap, mức tăng tuyệt đối về test accuracy là 19,00%; so với ViTOverlap là 13,31%; và so với HybridCNNChannelTokens là 8,3%.

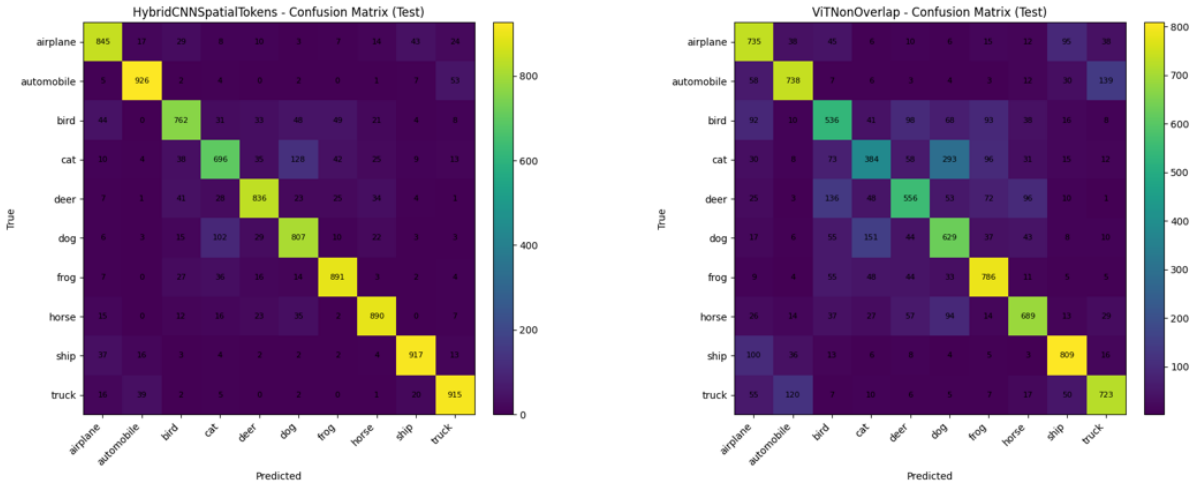
Nguyên nhân hợp lý là ở CIFAR-10, đặc trưng cục bộ như cạnh, texture và hình dạng ngắn hạn vẫn rất quan trọng. Backbone CNN giúp trích xuất những tín hiệu cục bộ này tốt hơn ngay từ đầu. Sau đó, Transformer chỉ cần xử lý chuỗi gồm 64 token không gian để học quan hệ toàn cục giữa các vùng ảnh. Cách kết hợp này tận dụng được ưu điểm của cả CNN và Transformer.

Khi so sánh hai cách patch embedding trực tiếp từ ảnh, ViTOverlap cho kết quả

tốt hơn ViTNonOverlap. Cụ thể, test accuracy tăng từ 65,85% lên 71,54%, tức tăng 5,69%. Điều này cho thấy việc để các patch chồng lắp lên nhau giúp mô hình giữ được thêm thông tin biên giữa các vùng lân cận. Đổi lại, số token tăng từ 64 lên 225, kéo theo số tham số tăng thêm 15.456 và thời gian trung bình mỗi epoch tăng khoảng 6,18 giây.

So sánh hai mô hình hybrid cũng cho một kết luận đáng chú ý: dùng token theo chiều không gian H×W hiệu quả hơn rõ rệt so với dùng token theo chiều kênh C. HybridCNNChannelTokens vẫn tốt hơn hai mô hình ViT thuần, nhưng vẫn thấp hơn HybridCNNSpatialTokens 8,35 điểm phần trăm. Điều này gợi ý rằng với CIFAR-10, việc giữ cấu trúc không gian của feature map dưới dạng chuỗi token là tự nhiên hơn và thuận lợi hơn cho bài toán phân loại ảnh.

Về chi phí tính toán, ViTNonOverlap là mô hình gọn nhất và nhanh nhất: chỉ 347.722 tham số và 23,65 giây mỗi epoch. Tuy nhiên đây cũng là mô hình có độ chính xác thấp nhất. Vì vậy, nếu ưu tiên tốc độ và tài nguyên thì ViTNonOverlap là baseline hợp lý; còn nếu ưu tiên hiệu năng, HybridCNNSpatialTokens là lựa chọn tốt nhất trong bốn mô hình đã thử.



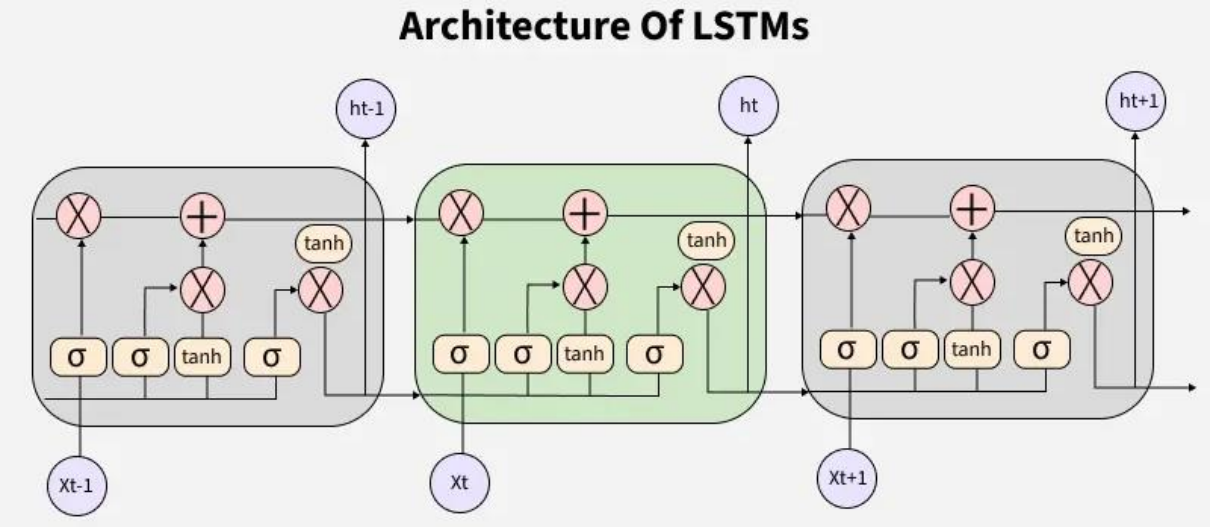
Hình 19. Confusion matrix của mô hình tốt nhất và yếu nhất

Confusion matrix cho thấy mô hình tốt nhất vẫn gặp khó ở các cặp lớp có ngữ nghĩa gần nhau như cat–dog, airplane–ship hoặc automobile–truck, nhưng mức nhầm lẫn đã giảm nhiều so với ViTNonOverlap. Ví dụ, ở HybridCNNSpatialTokens, lớp cat bị dự đoán nhầm thành dog 128 lần; trong khi ở ViTNonOverlap con số này tăng lên 293 lần. Điều này củng cố nhận định rằng hybrid CNN + Transformer học được đặc trưng cục bộ tốt hơn nên phân tách lớp hiệu quả hơn.

Qua thực nghiệm trên CIFAR-10, mô hình HybridCNNSpatialTokens là phương án tốt nhất trong bốn kiến trúc được khảo sát. Kết quả này cho thấy việc dùng CNN để trích xuất đặc trưng cục bộ, sau đó dùng Transformer để mô hình hoá quan hệ toàn cục giữa các vùng đặc trưng, là hướng phù hợp cho ảnh kích thước nhỏ như CIFAR-10. Ngoài ra, patch overlap có cải thiện đáng kể so với patch không overlap, còn biểu diễn token theo chiều không gian hiệu quả hơn biểu diễn token theo chiều kênh.

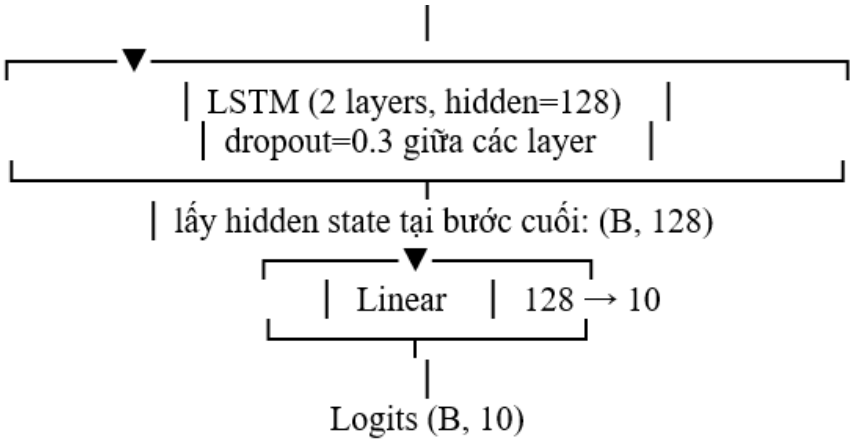
V. Mô hình phân loại dựa trên LSTM/GRU

1. Mô hình chung: LSTMClassifier



Hình 20. Kiến trúc tổng quan LSTM

Cả hai cách biểu diễn đều dùng chung một kiến trúc LSTM, chỉ khác nhau ở tham số input_size:



Bảng 4.*Thông số huấn luyện mô hình LTSM*

Tham số	Giá trị
hidden_size	128
num_layers	2
dropout	0.3 (giữa các LSTM layer)
num_classes	10
Đầu ra cuối	Hidden state tại bước thời gian cuối cùng

Hai mô hình có quy mô tham số tương đối tương đồng nhưng vẫn có sự khác biệt nhất định. Cụ thể, mô hình Row-wise có khoảng 600 nghìn tham số, trong khi mô hình Patch-wise nhẹ hơn với khoảng 530 nghìn tham số. Sự chênh lệch này cho thấy mô hình Patch-wise được thiết kế gọn hơn, giúp giảm chi phí tính toán và có thể cải thiện tốc độ huấn luyện, trong khi mô hình Row-wise với số lượng tham số lớn hơn có tiềm năng biểu diễn mạnh hơn nhưng đi kèm với độ phức tạp cao hơn.

2. Xử lý dữ liệu (Data Processing & DataLoader)

a) Tập dữ liệu

CIFAR-10: 60.000 ảnh RGB kích thước 32×32, 10 lớp, với tập train: 50.000 ảnh và test: 10.000 ảnh

b) Hai cách biểu diễn ảnh thành chuỗi

Ảnh CIFAR-10 có kích thước (3, 32, 32) được chuyển thành sequence theo hai cách:

- Cách 1 — Row-wise (từng hàng pixel):

  (B, 3, 32, 32) → permute(0,2,3,1) → (B, 32, 32, 3) → reshape → (B, 32, 96)

  + seq_len = 32 (mỗi hàng pixel là 1 bước thời gian)
  + input_size = 96 (32 pixel × 3 kênh màu)
  + LSTM học quan hệ theo chiều dọc giữa các hàng ảnh

- Cách 2 — Patch-wise (chia ô 4×4)

  (B, 3, 32, 32) → chia thành lưới 8×8 patch, mỗi patch 4×4 pixel → (B, 64, 48)

  + seq_len = 64 (8×8 = 64 patch)

+ input_size = 48 (4×4 pixel × 3 kênh màu)

+ Mỗi bước thời gian là một vùng cục bộ 2D của ảnh, đọc theo thứ tự raster (trái→phải, trên→dưới)

3. Quá trình huấn luyện

Bảng 5. *Bảng các siêu thông số của quá trình huấn luyện*

Tham số	**Giá trị**
Optimizer	Adam
Learning rate	0.001
Loss function	CrossEntropyLoss
Batch size	128
Max epochs	30
LR Scheduler	StepLR (step=5, γ=0.5)
Early stopping patience	2

4. Kết quả và Đánh giá

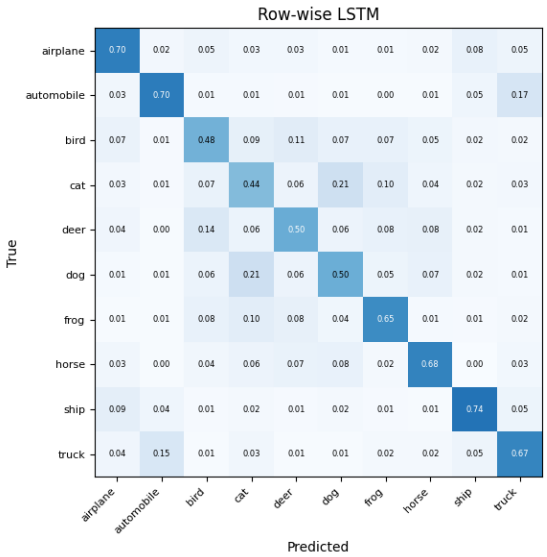
a) Tổng quan

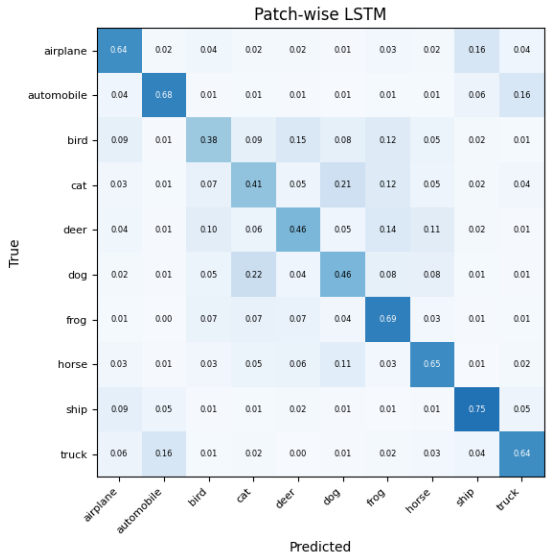
Mô hình	**seq_len**	**input_size**	**Epochs thực tế**	**Best Test Acc**	**Train Time**
Row-wise	32	96	15 / 30	**60.73%**	27.9s
Patch-wise	64	48	19 / 30	**57.33%**	35.4s

b) Các chỉ số đánh giá (trên tập test)

Chỉ số	**Row-wise**	**Patch-wise**
Accuracy	**60.6%**	57.5%
Precision (macro)	**0.6057**	0.5716
Recall (macro)	**0.6059**	0.5753
F1-score (macro)	**0.6055**	0.5715

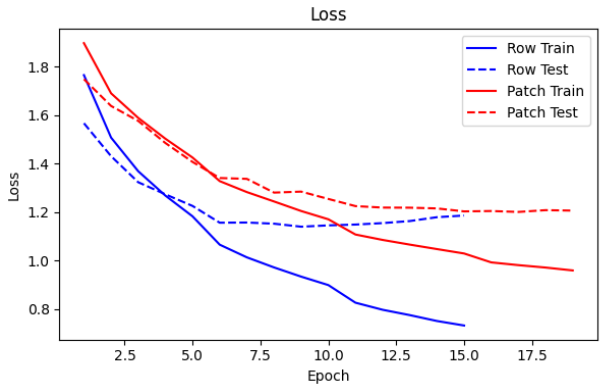
c) Kết quả theo từng lớp

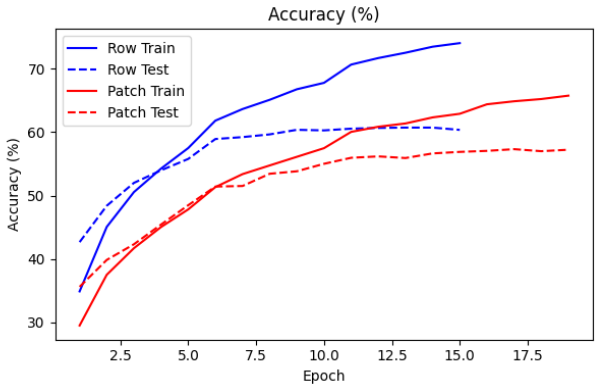




- Cả hai mô hình đều nhầm lẫn nhiều nhất giữa cat↔dog và bird↔deer — đây là các cặp lớp có đặc trưng thị giác tương tự.
- Ship và automobile là các lớp dễ phân loại nhất (nền đơn giản, hình dạng đặc trưng).
- Row-wise đặc biệt tốt hơn ở các lớp động vật (bird, deer, horse).

d) Quá trình hội tụ





Nhận xét hội tụ:

- Row-wise hội tụ nhanh hơn (đạt 55% test acc ở epoch 5 vs epoch 10 của Patch-wise).
- Cả hai đều có dấu hiệu overfitting nhẹ: train acc tiếp tục tăng trong khi test acc plateau.
- Patch-wise cần nhiều epoch hơn để hội tụ do sequence dài hơn (64 bước).

5. Nhận xét và Kết luận

a) Tại sao Row-wise tốt hơn Patch-wise?

Trái với kỳ vọng ban đầu (patch giữ thông tin cục bộ 2D nên có thể tốt hơn), Row-wise cho kết quả tốt hơn ~3%. Lý do:

- Gradient vanishing: Sequence dài hơn (64 vs 32 bước) khiến LSTM khó truyền gradient về các bước đầu, đặc biệt ở num_layers=2.
- Input size nhỏ hơn: Mỗi bước của Patch-wise chỉ có 48 chiều (vs 96 của Row-wise), mang ít thông tin hơn mỗi bước.
- Tốc độ hội tụ: Row-wise hội tụ nhanh hơn → ít bị ảnh hưởng bởi learning rate decay (StepLR).

b) Hạn chế chung của LSTM trên ảnh

- LSTM xử lý dữ liệu tuần tự, không khai thác được tính locality 2D và spatial invariance vốn là thế mạnh của CNN.
- Accuracy ~57–61% thấp hơn đáng kể so với CNN đơn giản (~85–90%) trên CIFAR-10.
- Để cải thiện: dùng Bidirectional LSTM, tăng hidden_size, hoặc kết hợp CNN + LSTM.

TÀI LIỆU THAM KHẢO

[1] I. Goodfellow, Y. Bengio, and A. Courville, Deep Learning. Cambridge, MA, USA: MIT Press, 2016.

[2] A. Krizhevsky, V. Nair, and G. Hinton, "Learning multiple layers of features from tiny images," University of Toronto, Tech. Rep., 2009

[3] A. Vaswani et al., "Attention is all you need," in Advances in Neural Information Processing Systems, 2017, vol. 30.

[4] S. Hochreiter and J. Schmidhuber, "Long short-term memory," Neural Comput., vol. 9, no. 8, pp. 1735–1780, 1997.

[5] K. Cho et al., "Learning phrase representations using RNN encoder-decoder for statistical machine translation," in Proc. Conf. on Empirical Methods in Natural Language Processing (EMNLP), 2014, pp. 1724–1734.